

Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: minhaz.ahmed@gmail.com

Content

- ▶ Pre-training
- ▶ Transfer learning
- ▶ Advantages
- ▶ Types of Transfer learning
- ▶ Transfer learning in NLP
- ▶ Training
- ▶ Summary

Pre-training

- Training a model from scratch
- Weights are randomly initialized
- Model is then pre-trained on large amounts of data

Pre-training :BERT

Item	BERT
Year	2018
Number of parameters	109M
Training time	12 days
Infrastructure	8 x V100 GPUs (*)
Size of dataset used for training	16GB
Training tokens (dataset)	250B
Dataset source	Wikipedia
	BookCorpus

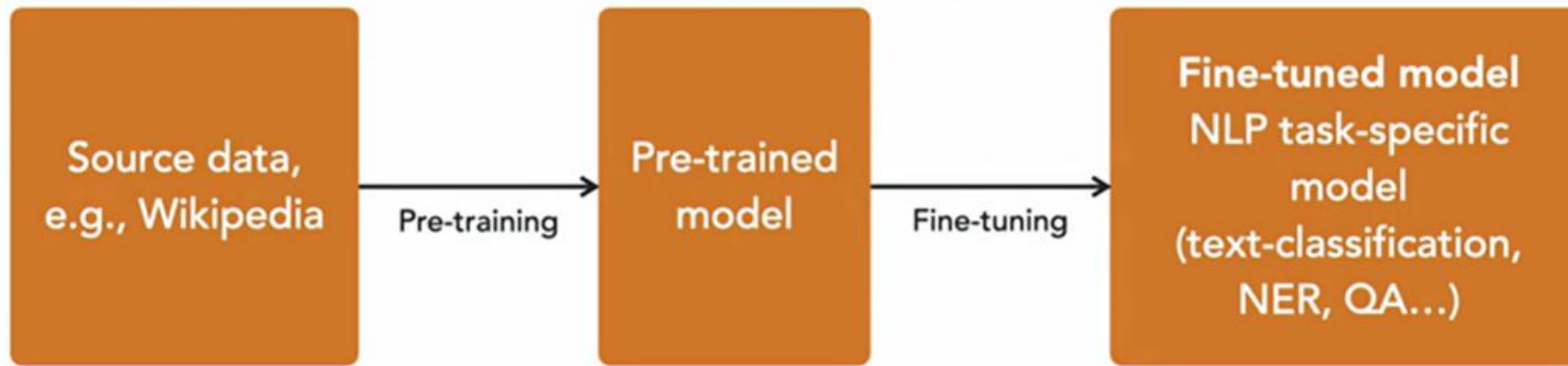
Pre-training :RoBERTa

Item	BERT	RoBERTa
Year	2018	2019
Number of parameters	109M	125M
Training time	12 days	1 day
Infrastructure	8 x V100 GPUs (*)	1024 V100 GPUs
Size of dataset used for training	16GB	160GB
Training tokens (dataset)	250B	2000B
Dataset source	Wikipedia	Wikipedia
	BookCorpus	BookCorpus
		Common Crawl news dataset
		OpenWebText
		Common Crawl stories

Pre-training : GPT3

Item	BERT	RoBERTa	GPT-3
Year	2018	2019	2020
Number of parameters	109M	125M	174,600M (175B)
Training time	12 days	1 day	34 days
Infrastructure	8 x V100 GPUs (*)	1024 V100 GPUs	10,000 V100 GPUs
Size of dataset used for training	16GB	160GB	4500GB
Training tokens (dataset)	250B	2000B	300B
Dataset source	Wikipedia	Wikipedia	Wikipedia
	BookCorpus	BookCorpus	Common Crawl
		Common Crawl news dataset	WebText2
		OpenWebText	Books1
		Common Crawl stories	Books2

Fine-Tuning

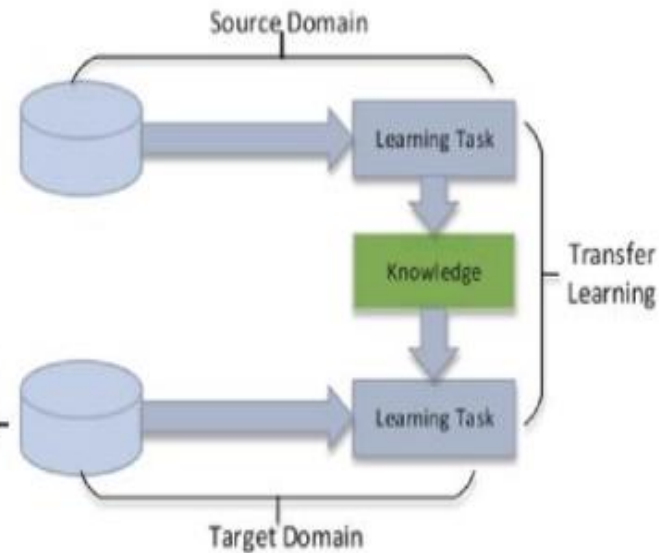


Transfer Learning

- Transfer learning aims to leverage the learned knowledge from a resource-rich domain/task to help learning a task with not sufficient training data.
 - Sometimes referred as domain adaptation

- The resource-rich domain is known as the **source** and the low-resource task is known as the **target**.

- Transfer learning works the best if the model features learned from the source task are **general** (i.e., domain-independent).



Transfer learning example in practice

- **Image Processing:** Learning a character recognition system on English and use it for German character recognition.
 - Which language has more labeled characters?
 - Do characters in these two languages share common traits?
- **Sentiment Analysis:** Learning a sentiment analysis system on Amazon's laptops review and apply it to digital camera review.
 - Should we manually label many camera reviews from scratch?
- What are source and target tasks/domain in the above examples?

Benefits of Transfer learning

- ▶ Faster development
- ▶ Less data to fine-tune.
- ▶ Good result

Transfer learning definition

- Let $\mathcal{D}_s$ and $\mathcal{D}_t$ denote the source and target domains, respectively.
 - A domain contains the feature space.
- Also let $\mathcal{T}_s$ and $\mathcal{T}_t$ be the source and target tasks, respectively.
- Transfer learning aims to help improve the learning of the target predictive function $f_T(.)$ in $\mathcal{D}_t$ using the knowledge in $\mathcal{D}_s$ and $\mathcal{T}_s$, where $\mathcal{D}_s \neq \mathcal{D}_t$ and/or $\mathcal{T}_s \neq \mathcal{T}_t$.

Deep Transfer learning

- The recent progress in deep learning has facilitated transfer learning mainly because of two reasons:
 1. Networks can be pre-trained on one domain and be tuned on another domain.
 2. Network weights can be shared among different tasks.
- A transfer learning task $\langle \mathcal{D}_s, \mathcal{T}_s, \mathcal{D}_t, \mathcal{T}_t, f_T(\cdot) \rangle$ is a deep transfer learning task where $f_T(\cdot)$ is a non-linear function defined by a deep neural network.

Types of Deep Transfer learning

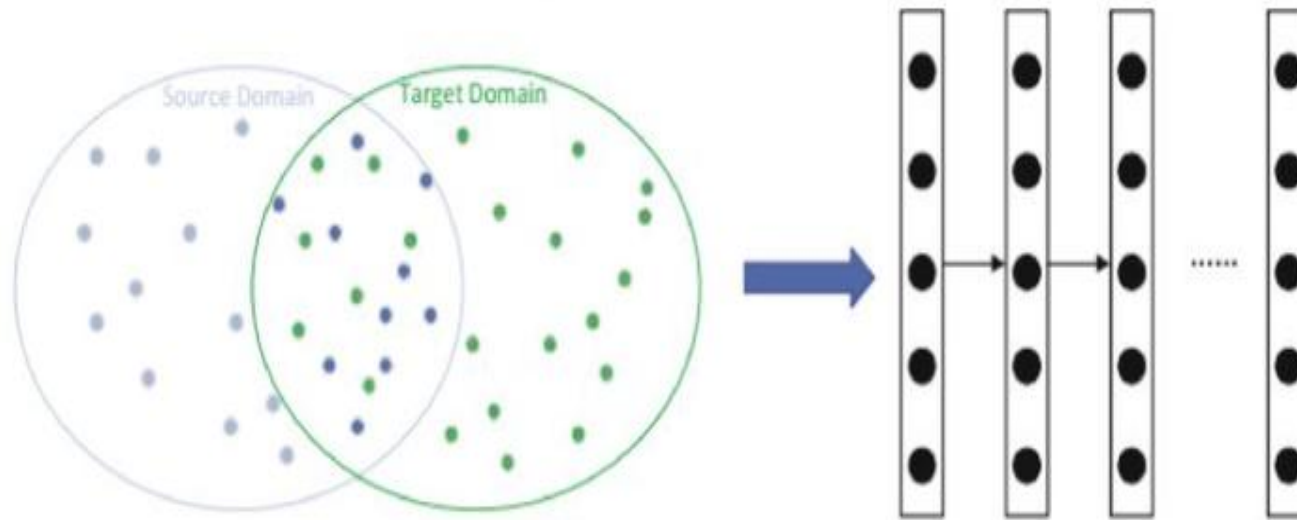
- There are Four major types of deep transfer learning:
 - **Instance-based:** Reusing the instances in source domain by assigning appropriate weight to them
 - **Representation-based:** Mapping instances from two domains into a new feature space with higher similarity of source and domain
 - **Model Sharing-based:** Reusing part of a network (or its parameters) that is pre-trained in the source domain
 - **Adversarial-based:** Using adversarial learning to find transferable features that are suitable for both domains

Instance based deep transfer learning

- **Assumption:** 'Some' instances in the source domain can be utilized in the target domain with appropriate weights.
- **Approach:**
 1. Filtering out the instances that are not similar to the target.
 2. Re-weighting instances in source domain to obtain a distribution similar to target domain.
 3. Training a deep neural network on the collection of re-weighted instances as well as labeled instances (if any) in the target domain.

Instance based deep transfer learning

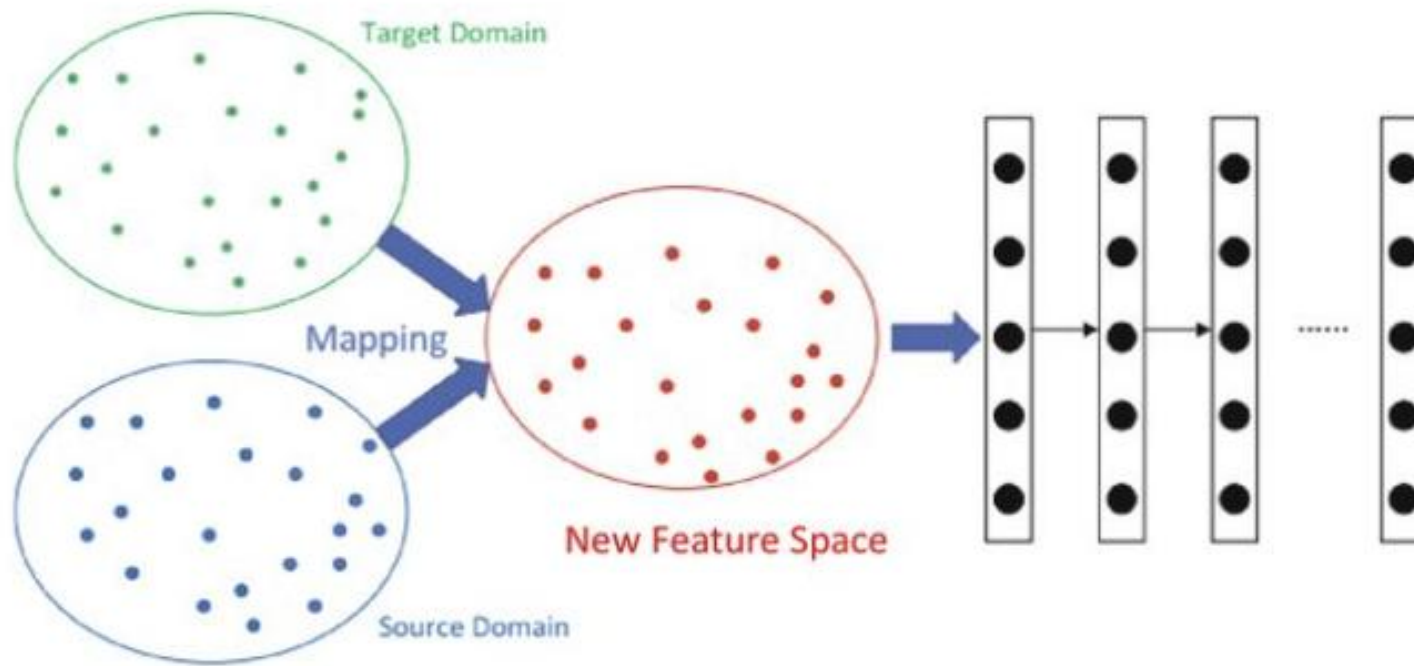
- Instances shown in light blue in the source domain are not similar to the target domain and therefore are excluded from the training dataset.
- Similar instances in the source domain (dark blue) are included in the training dataset with appropriate weights.



Representation based deep transfer learning

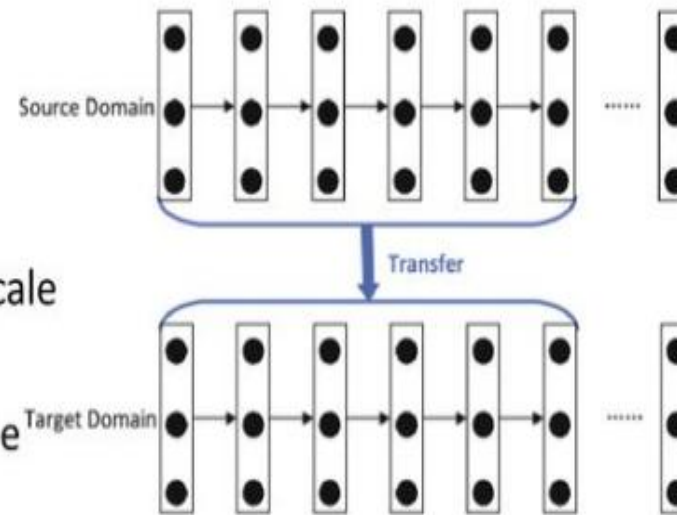
- **Assumption:** There exists a new feature space/representation in which similar instances from source and target domain are close to each other.
- **Approach:**
 1. Instances from source and target domain are mapped into a new feature space in which similar instances are close to each other.
 2. Instances in the new feature space can be used as the training set of the neural network.

Representation based deep transfer learning



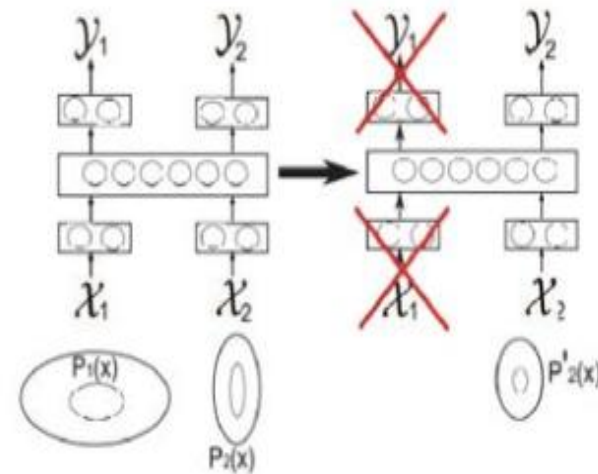
Model sharing based deep transfer learning

- **Assumption:** The layers of a trained model can be treated as feature extractor to extract high-level features from a new domain.
 - Most common type of transfer learning
- **Approach:**
 1. Network is trained in source domain with large-scale training dataset.
 2. Part/all of the pre-trained network is reused in the architecture of the model for target model.
 3. Weights of the reused part can be updated through training on the target domain (aka fine-tuning).



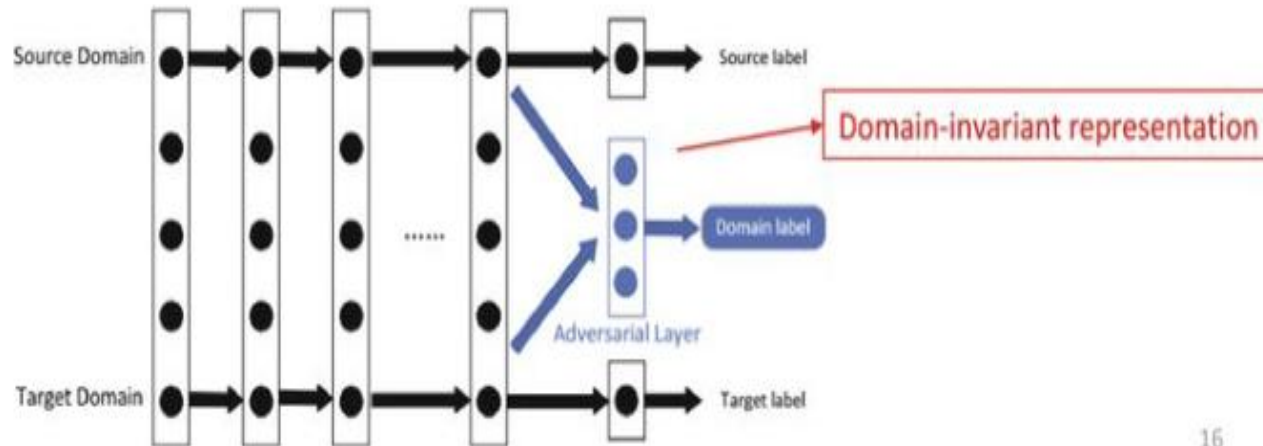
Model sharing based deep transfer learning

- It is also possible that the entire model is shared and trained on source and target domains **jointly**.
- $\mathcal{X}_1$ and $\mathcal{Y}_1$: feature and label spaces for the learning task in the source domain
- $\mathcal{X}_2$ and $\mathcal{Y}_2$: feature and label spaces for the learning task in the target domain.
- At the runtime, only the target domain is concerned.



Adversarial based deep transfer learning

- **Assumption:** Good representation should be domain-invariant for source and target domain while it is discriminative for the main learning task.
- **Approach:**
 1. The model extracts features from two domains and sent them to the adversarial layer.
 2. The adversarial layer tries to discriminate the origin of the features.
 - Review the adversarial learning tutorial to refresh your memory.



Implementing model sharing based deep transfer learning



Available models

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4
MobileNetV2	14	71.3%	90.1%	3.5M	105	25.9	3.8
DenseNet121	33	75.0%	92.3%	8.1M	242	77.1	5.4
DenseNet169	57	76.2%	93.2%	14.3M	338	96.4	6.3
DenseNet201	80	77.3%	93.6%	20.2M	402	127.2	6.7
NASNetMobile	23	74.4%	91.9%	5.3M	389	27.0	6.7
NASNetLarge	343	82.5%	96.0%	88.9M	533	344.5	20.0
EfficientNetB0	29	77.1%	93.3%	5.3M	132	46.0	4.9
EfficientNetB1	31	79.1%	94.4%	7.9M	186	60.2	5.6
EfficientNetB2	36	80.1%	94.9%	9.2M	186	80.8	6.5
EfficientNetB3	48	81.6%	95.7%	12.3M	210	140.0	8.8
EfficientNetB4	75	82.9%	96.4%	19.5M	258	308.3	15.1



Pre-trained models

Implementation

- We build a Model sharing-based Deep Transfer Learning in Image processing which uses CNN as the core neural network model.
- We use VGG-16, a pre-trained CNN on more than a million images from ImageNet. ([available in Keras](#))

Determining the libraries

```
from keras import applications ← Contains VGG trained model in Keras
from keras.preprocessing.image import
ImageDataGenerator ← Keras Builtin library for preprocessing images
from keras import optimizers ← Contains different loss Functions used for BackProp
from keras.models import Sequential, Model
from keras.layers import Dropout, Flatten, ← Contains different type of layers
Dense, GlobalAveragePooling2D
from keras import backend as k
```


NLP example

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Flatten, Dense

# Example dataset
texts = ['Tashkent is a beautiful city!',
        'It has long traffic',
        'I loved the city park.',
        'Markets are crowded.',
        'The infrastructure were well-developed.']
labels = [1, 0, 1, 0, 1] # 1: Positive, 0: Negative

# Split the dataset into train and test sets
train_texts, test_texts, train_labels, test_labels = train_test_split(texts, labels, test_size=0.2, random_state=42)

# Tokenize the texts
tokenizer = Tokenizer()
tokenizer.fit_on_texts(train_texts)
train_sequences = tokenizer.texts_to_sequences(train_texts)
test_sequences = tokenizer.texts_to_sequences(test_texts)
```

NLP example

```
# Load pre-trained GloVe word embeddings
embedding_dim = 100
embeddings_index = {}
with open('glove.6B.100d.txt', encoding='utf-8') as f:
    for line in f:
        word, coefs = line.split(maxsplit=1)
        coefs = np.fromstring(coefs, 'f', sep=' ')
        embeddings_index[word] = coefs

# Create embedding matrix
word_index = tokenizer.word_index
num_words = len(word_index) + 1
embedding_matrix = np.zeros((num_words, embedding_dim))
for word, i in word_index.items():
    embedding_vector = embeddings_index.get(word)
    if embedding_vector is not None:
        embedding_matrix[i] = embedding_vector
```

NLP example

```
# Build the model
model = Sequential([
    Embedding(num_words, embedding_dim, weights=[embedding_matrix], input_length=maxlen, trainable=False),
    Flatten(),
    Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
model.fit(train_sequences, train_labels, epochs=10, batch_size=1)

# Evaluate the model
loss, accuracy = model.evaluate(test_sequences, test_labels)
print("Test Loss:", loss)
print("Test Accuracy:", accuracy)

# Make predictions
predictions = model.predict(test_sequences)
binary_predictions = (predictions > 0.5).astype(int).flatten()
print("Test Accuracy (binary predictions):", accuracy_score(test_labels, binary_predictions))
```

GloVe: Global Vectors for Word Representation

- ▶ GloVe is an unsupervised learning algorithm for obtaining vector representations for words. Training is performed on aggregated global word-word co-occurrence statistics from a corpus, and the resulting representations showcase interesting linear substructures of the word vector space.
- ▶ Derive the semantic relationship between words using word-word co-occurrence matrix

GloVe: Global Vectors for Word Representation

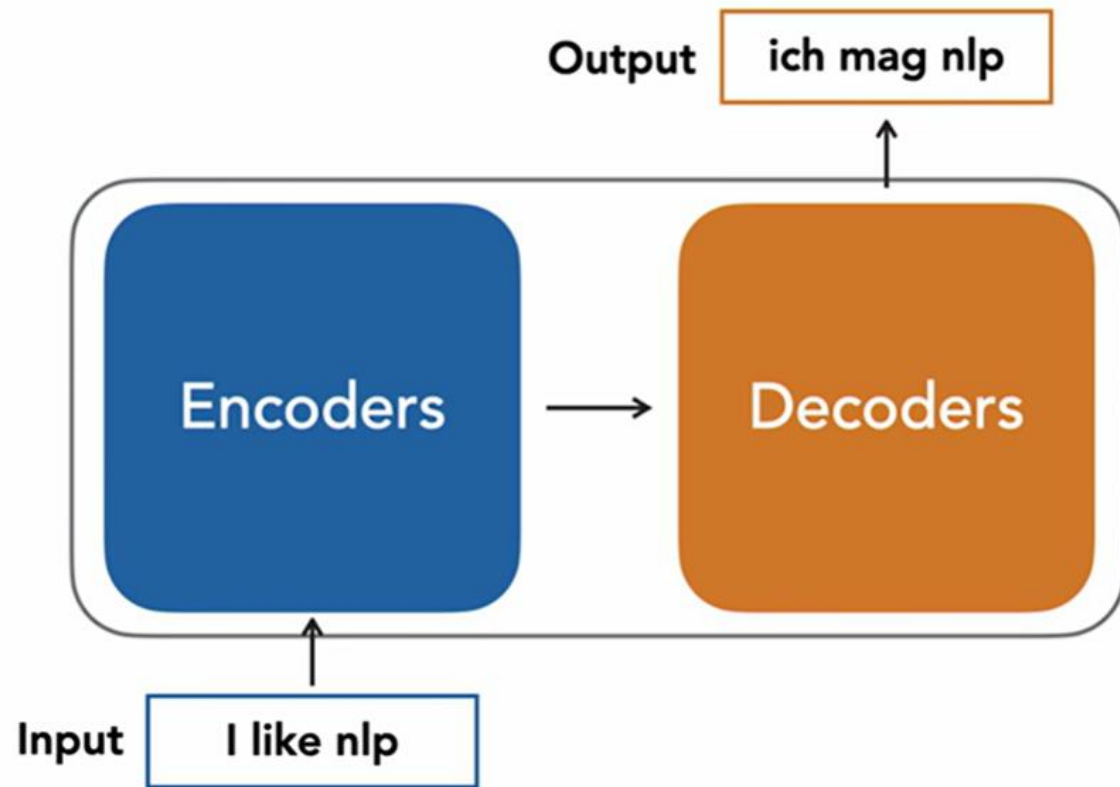
Token

```
the -0.038194 -0.24487 0.72812 -0.39961 0.083172 0.043953 -0.39141 0.3344 -0.57545 0.087459 0.28787 -0.06731 0.30906 -0.26384 -0.13231 -0.20757
0.33395 -0.33848 -0.31743 -0.48336 0.1464 -0.37304 0.34577 0.052041 0.44946 -0.46971 0.02628 -0.54155 -0.15518 -0.14107 -0.039722 0.28277 0.14393 0.23464 -0.31021
0.086173 0.20397 0.52624 0.17164 -0.082378 -0.71787 -0.41531 0.20335 -0.12763 0.41367 0.55187 0.57908 -0.33477 -0.36559 -0.54857 -0.062892 0.26584 0.30205
0.99775 -0.80481 -3.0243 0.01254 -0.36942 2.2167 0.72201 -0.24978 0.92136 0.034514 0.46745 1.1079 -0.19358 -0.074575 0.23353 -0.052062 -0.22044
0.057162 -0.15806 -0.30798 -0.41625 0.37972 0.15006 -0.53212 -0.2055 -1.2526 0.071624 0.70565 0.49744 -0.42063 0.26148 -1.538 -0.30223 -0.073438 -0.28312
0.57104 -0.25217 0.016215 -0.017099 -0.38984 0.87424 -0.72569 -0.51058 -0.52028 -0.1459 0.8278 0.27062
, -0.10767 0.1103 0.59812 -0.54361 0.67396 0.10663 0.038867 0.35481 0.06351 -0.094189 0.15786 -0.81665 0.14172 0.21939 0.58505 -0.52158 0.22783 -0.16642 -0.68228
0.3587 0.42568 0.19021 0.91963 0.57555 0.46185 0.42363 -0.095399 -0.42749 -0.16567 -0.056842 -0.29595 0.26037 -0.26606 -0.070404 -0.27662 0.15821 0.69825 0.43081
-0.33801 -0.58184 0.22364 -0.5778 -0.26862 -0.20425 0.56394 -0.58524 -0.14365 -0.64218 0.0054697 -0.35248 0.16162
1.1796 -0.47674 -2.7553 -0.1321 -0.047729 1.0655 1.1034 -0.2208 0.18669 0.13177 0.15117 0.7131 -0.35215 0.91348 0.61783 0.70992
0.23955 -0.14571 -0.37859 -0.045959 -0.47368 0.2385 0.20536 -0.18996 0.32507 -1.1112 -0.36341 0.98679 -0.084776 -0.54008 0.11726 -1.0194 -0.24424 0.12771 0.013884
0.080374 -0.35414 0.34951 -0.7226 0.37549 0.4441 -0.99059 0.61214 -0.35111 -0.83155 0.45293 0.082577
. -0.33979 0.20941 0.46348 -0.64792 -0.38377 0.038034 0.17127 0.15978 0.46619 -0.019169 0.41479 -0.34349 0.26872 0.04464 0.42131 -0.41032 0.15459 0.022239 -0.64653
0.25256 0.043136 -0.19445 0.46516 0.45651 0.68588 0.091295 0.21875 -0.70351 0.16785 -0.35079 -0.12634 0.66384 -0.2582 0.036542 -0.13605 0.40253 0.14289
0.38132 -0.12283 -0.45886 -0.25282 -0.30432 -0.11215 -0.26182 -0.22482 -0.44554 0.2991 -0.85612 -0.14503 -0.49086 0.0082973 -0.17491 0.27524
1.4401 -0.21239 -2.8435 -0.27958 -0.45722 1.6386 0.78808 -0.55262 0.65 0.086426 0.39012 1.0632 -0.35379 0.48328 0.346 0.84174 0.098707 -0.24213 -0.27053
0.045287 -0.40147 0.11395 0.0062226 0.036673 0.018518 -1.0213 -0.20806 0.64072 -0.068763 -0.58635
0.33476 -1.1432 -0.1148 -0.25091 -0.45907 -0.096819 -0.17946 -0.063351 -0.67412 -0.068895 0.53604 -0.87773 0.31802 -0.39242 -0.23394 0.47298 -0.028803
of -0.1529 -0.24279 0.89837 0.16996 0.53516 0.48784 -0.58826 -0.17982 -1.3581 0.42541 0.15377 0.24215 0.13474 0.41193 0.67043 -0.56418 0.42985 -0.012183 -0.11677
0.31781 0.054177 -0.054273 0.35516 -0.30241 0.31434 -0.33846 0.71715 -0.26855 -0.15837 -0.47467 0.051581 -0.33252 0.15003 -0.1299 -0.54617 -0.37843 0.64261
0.82187 -0.08000 0.078479 -0.96976 -0.57741 0.56491 -0.39873 -0.057099 0.19743 0.065706 -0.48092 -0.20125 -0.40834 0.39456 -0.02642 -0.11838
1.012 -0.53171 -0.7474 -0.042981 -0.74849 1.7574 0.59085 0.04885 0.78267 0.38497 0.42097 0.67882 0.10337 0.6328 -0.026595 0.58647 -0.44332 0.33057 -0.12022 -0.55645
0.073611 0.20915 0.43395 -0.012761 0.089874 -1.7991 0.084808 0.77112 0.63105 -0.90685 0.60326 -1.7515 0.18596 -0.50687 -0.70203 0.66578 -0.81304
0.12012 0.020103 -0.26757 0.727 -0.59363 -0.34839 -0.56094 -0.591 1.0039 0.20664
to -0.1897 0.050024 0.19084 -0.049184 -0.089737 0.21006 -0.54952 0.098377 -0.20135 0.34241 -0.092677 0.161 -0.13268 -0.2816 0.18737 -0.42959 0.96039 0.13972 -1.0781
0.40518 0.50539 -0.55064 0.4844 0.38044 -0.0029055 -0.34942 -0.099696 -0.78368 1.0363 -0.2314 -0.47121 0.57126 -0.21454 0.35958 -0.48319 1.0875 0.28524
0.12447 -0.039248 -0.076732 -0.76343 -0.32409 -0.5749 -1.0893 -0.41811 0.4512 0.12112 -0.51367 -0.13349 -1.1378 -0.28768 0.16774 0.55804 1.5387
0.018859 -2.9721 -0.24216 -0.92495 2.1992 0.28234 -0.3478 0.51621 -0.43387 0.36852 0.74573 0.072102 0.27931
0.92569 -0.050336 -0.85856 -0.1358 -0.92551 -0.33991 -1.0394 -0.067203 -0.21379 -0.4769 0.21377 -0.84008 0.052536 0.59298 0.29604 -0.67644 0.13916 -1.5504 -0.20765
0.7222 0.52056 -0.076221 -0.15194 -0.13134 0.058617 -0.31869 -0.61419 -0.62393 -0.41548 -0.038175 -0.39804 0.47647 -0.15983
```

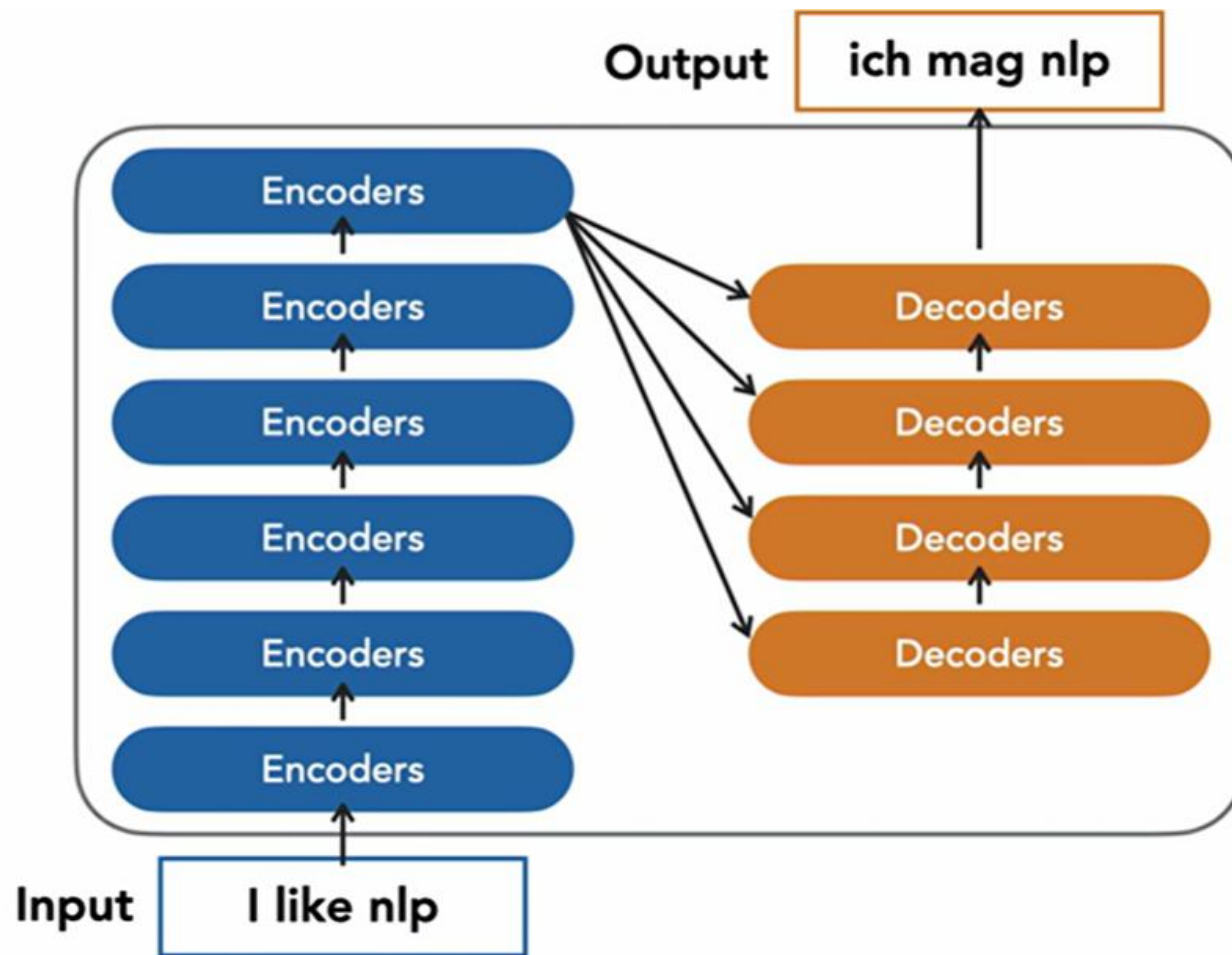
Scaling up transformers

Model	Layers	Width	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	
BERT-Large	24	1024	16	340M	13 GB	
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)

Transformer overview



Transformer overview

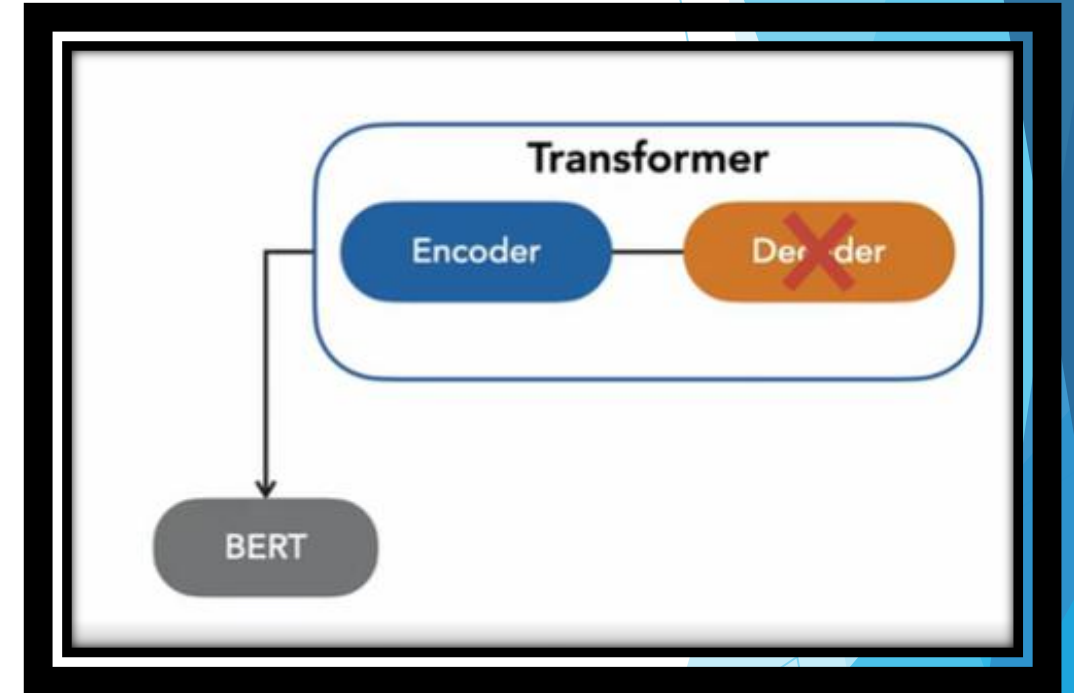


Popular Encoder decoder model

- Generative tasks
- BART (Bidirectional and Auto-Regressive Transformer)
- T5 T5 is an encoder-decoder model and converts all NLP problems into a text-to-text format.

Encoder only model

- ▶ Understanding of input
 - ▶ Sentence classification
 - ▶ Named entity recognition.
-
- ▶ Family of BERT model
 - ▶ BERT
 - ▶ RoBERTa
 - ▶ DISTIL BIRT



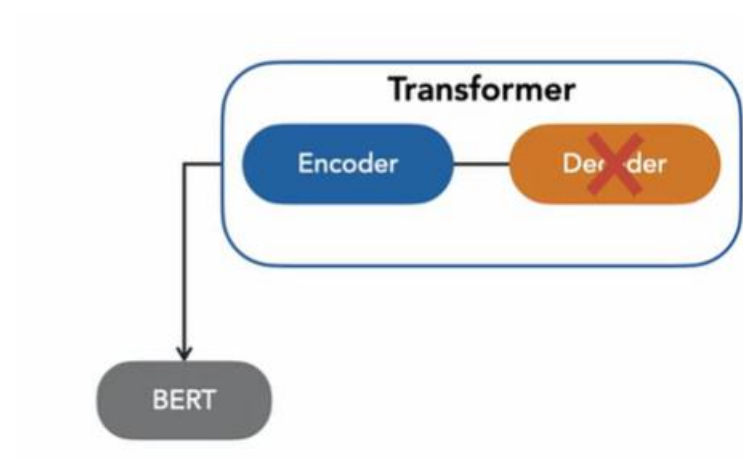
BERT does not do

- ▶ BERT does not do generate text.
- ▶ TEXT translation.
- ▶ TEXT summarization.

BERT can do

Does:

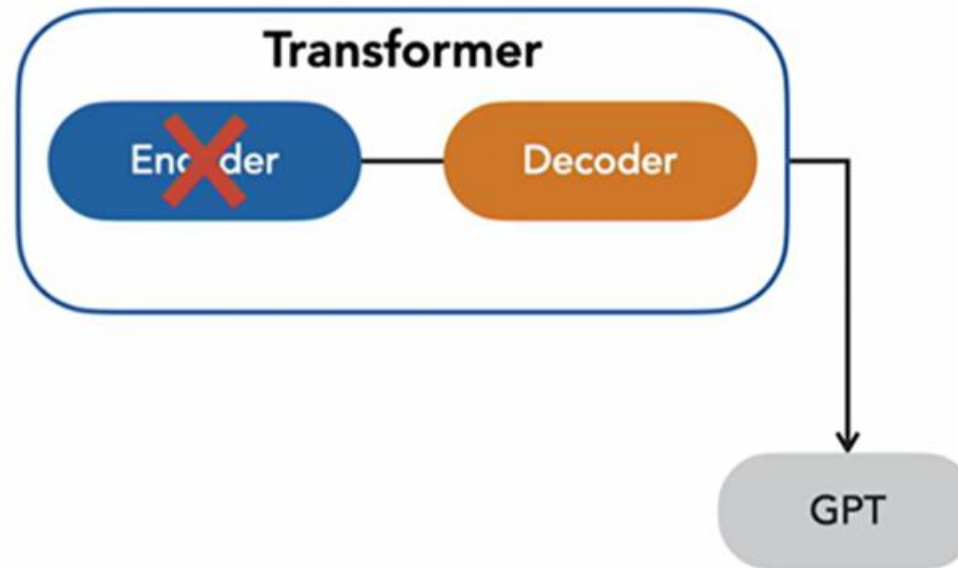
- Text classification
- Named entity recognition
- Question answering
- Fill in the blank



Decoder only model

- ▶ Good for generating task

- ▶ Such as
- ▶ GPT
- ▶ GPT2
- ▶ GPT3



A transformer block

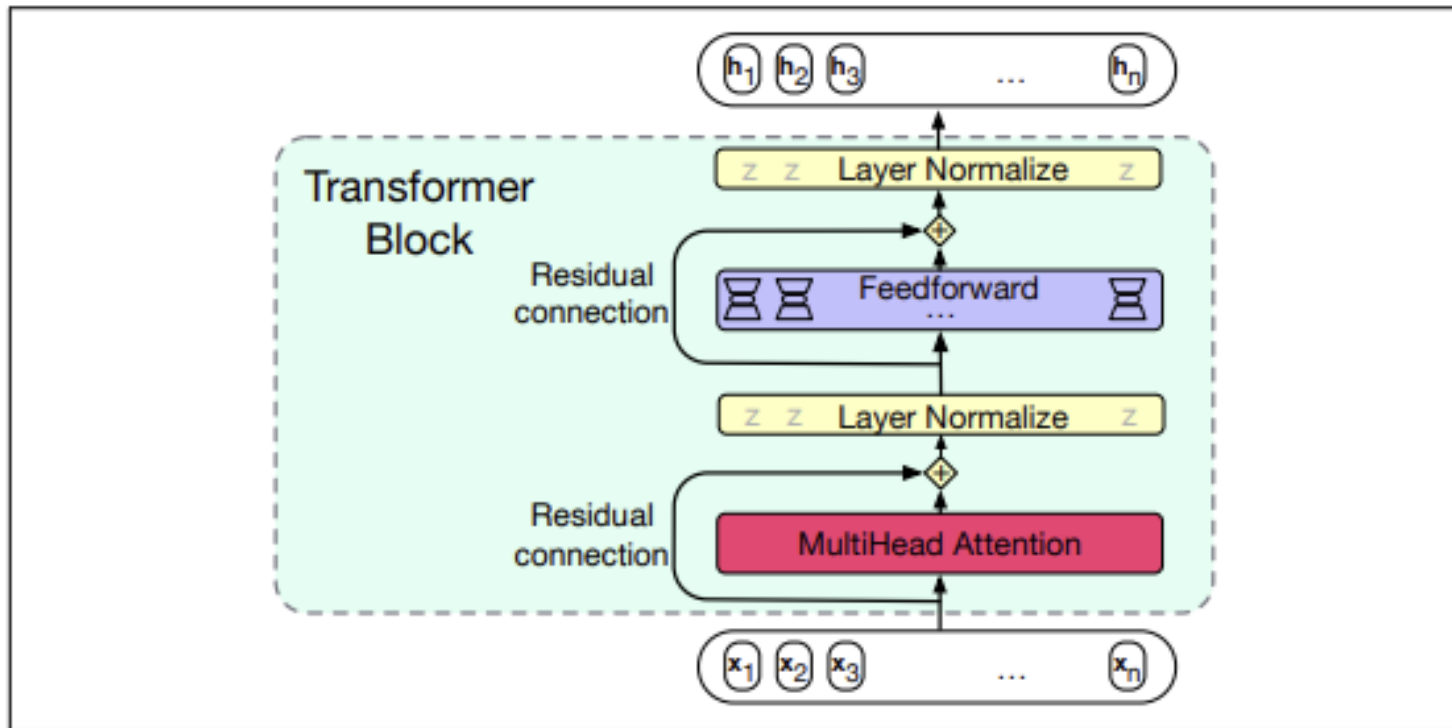
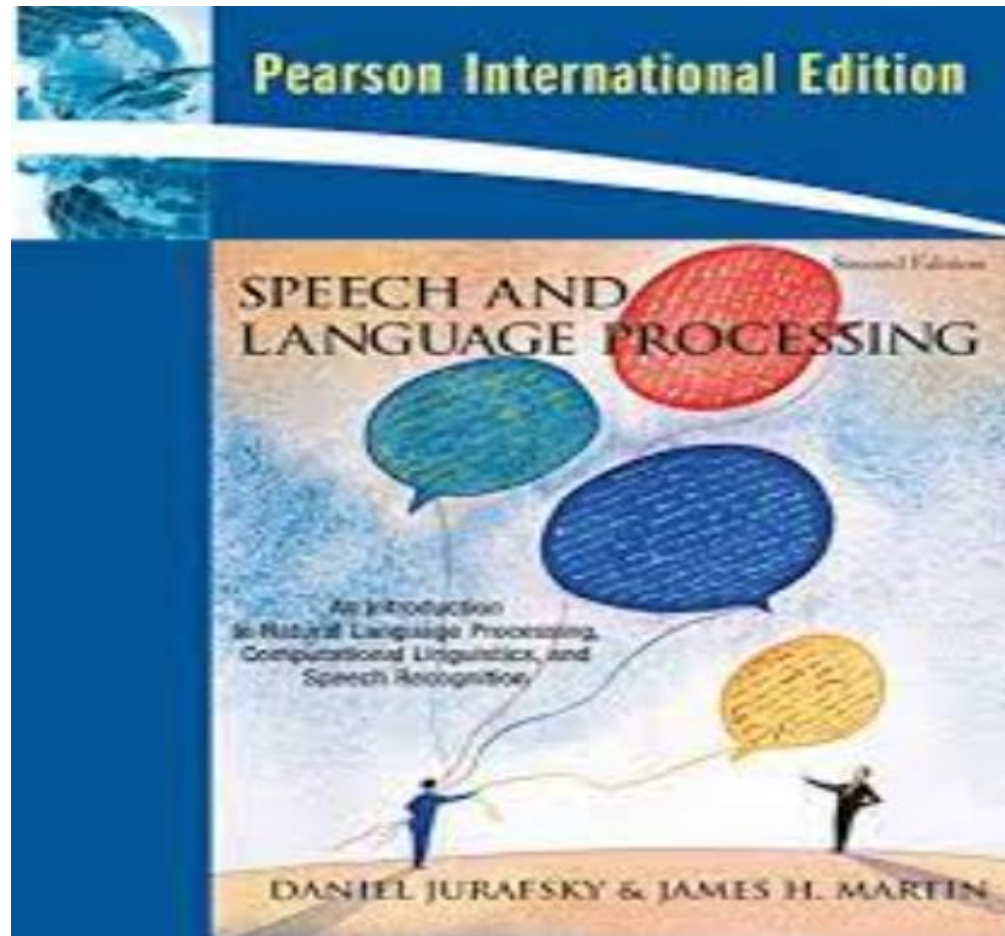


Figure 10.6 A transformer block showing all the layers.

Reference



Chapter 10

Question



Thank you

